

PYTHON FOR DATA SCIENCE • MODULE 1 • TOPIC 1.13

NumPy Statistical Functions

Day 14 — describing data, and coping when it is missing

90 min

Concepts + live code

30 min

Practice notebook

nan

The value that breaks sums



Boston University

Prepared & presented by **Srinivasa Sai Chava**

Before we start — five questions from Day 13

1 Do (3,4) and (3,) broadcast?

→ No. Align right: 4 against 3.

2 How do I make a (3,) a column?

→ `reshape(-1, 1)` — or `[:, None]`.

3 `m.sum(axis=0)` on a (2,3) — shape?

→ (3,) — the named axis disappears.

4 `a * b` or `a @ b` for a dot product?

→ `a @ b`. The `*` is element-wise.

5 What does `argmax` return?

→ A position, not the value.

Today is the last NumPy day. Everything here leans on yesterday's axis rule.

How today's two hours are structured

1

Centre

mean, median — and why they differ

20 min

2

Spread

std, var, percentiles, IQR

20 min

3

Missing data

nan, and the nan-aware functions

30 min

4

Relationships

correlation and covariance

15 min

5

Mini build

A summary report of a messy dataset

5 min

6

Practice notebook

You code, I walk around

30 min

Section 3 gets the most time. One missing value silently turns an entire average into nan.

By the end of today you will be able to

1

Summarise

Describe a dataset's centre and spread, and pick the right measure

2

Rank

Use percentiles and the IQR to find where a value sits

3

Survive

Detect missing values and compute around them

4

Relate

Measure how two variables move together — and not over-claim

A warning about this session

Every function today returns a number, and a number always looks authoritative. An average computed over data you have not examined can be confidently, precisely wrong — one outlier, or one missing value handled carelessly, is enough. Today is as much about knowing which summary to trust as about how to compute it.



Centre

Which average, and when • 20 minutes

- mean
- median
- Why they differ
- Weighted average
- No np.mode

The mean is not always the average you want

Six salaries, in thousands



↑ one outlier

The mean uses every value, so one extreme number drags it a long way. The median only cares about the middle position, so it barely moves.

```
mean    -> 110.17
median  -> 32.50
```

Five people earn about 32.
The mean describes nobody.

Which one to use

- Symmetric data, no extremes → mean
- Skewed data, or outliers → median
- Report BOTH when they disagree — the gap is itself information

Weighted average

```
marks    = np.array([88., 71., 64.])
weights  = np.array([0.5, 0.3, 0.2])

np.average(marks, weights=weights)
-> 78.1
```

np.average takes weights; np.mean does not.

There is no np.mode — here is why, and what to use

! np.mode does not exist

```
np.mode(x)
-> AttributeError
```

NumPy is a numerical array library, not a statistics package. The mode is awkward to define — a dataset can have two modes, or none at all — so NumPy leaves it out rather than guessing what you want.

Build it from np.unique

```
x = np.array([3, 1, 4, 1, 5, 1, 4])

vals, counts = np.unique(x, return_counts=True)
# vals    [1 3 4 5]
# counts  [3 1 2 1]

vals[counts.argmax()] -> 1
```

Day 13's argmax, giving the position of the largest count.

np.unique is worth knowing for its own sake

```
np.unique(x)           sorted, duplicates gone
np.unique(x, return_counts=True)  a frequency table
len(np.unique(x))       how many distinct values
```

This is Day 4's set, but sorted and countable — and it is how you check the distinct values in a column before trusting it. In two weeks pandas gives you `value_counts()`, which does exactly this and returns it already sorted by frequency.



Spread

How scattered is it · 20 minutes

- std & var
- ddof, again
- Percentiles
- IQR

Spread — two datasets with the same mean

Both average 50. They are not the same data.

A	48	49	50	51	52	std 1.41
B	10	30	50	70	90	std 28.28

```
var = std ** 2
std = in the SAME
      units as the data
```

⚠ NumPy is inconsistent about ddof

```
arr.std()      ddof=0  divides by n
arr.var()      ddof=0  divides by n
np.cov(a, b)   ddof=1  divides by n-1 !
```

std and var default to the population formula, but cov defaults to the sample one. That is NumPy's own inconsistency, not a typo. pandas defaults everything to ddof=1.

Range, quickly

```
arr.max() - arr.min()
np.ptp(arr)      the same thing
# 'peak to peak'
```

The range uses only two values, so a single outlier defines it completely. The IQR on the next slide is the robust alternative.

Percentiles — where does a value sit?

`a = np.arange(1, 11)`

the numbers 1 to 10



Call	Gives	Means
<code>np.percentile(a, 25)</code>	3.25	Q1 — a quarter of the data is below
<code>np.percentile(a, 50)</code>	5.5	the median, exactly
<code>np.percentile(a, 75)</code>	7.75	Q3 — three quarters below

The IQR — the robust measure of spread

```
q1, q3 = np.percentile(a, [25, 75])
iqr = q3 - q1          -> 4.5
# the spread of the MIDDLE HALF
```

Outliers sit outside it by definition, so they cannot distort it.

You will meet this again on Day 20

```
lower = q1 - 1.5 * iqr
upper = q3 + 1.5 * iqr
```

Anything outside those two bounds is the standard definition of an outlier — and it is exactly what draws the whiskers on a box plot. Two lines of arithmetic you already understand.

3

Missing Data

The first encounter with nan · 30 minutes

- What nan is
- How it spreads
- Detecting it
- The nan- functions

nan — the value that is not a value

Where it comes from

- A blank cell in a CSV
- A survey question nobody answered
- A sensor that failed for an hour
- $0/0$, or the log of a negative number

```
d = np.array([1., 2., np.nan, 4.])  
d.dtype -> float64    always a float
```

! nan is not equal to itself

```
np.nan == np.nan    -> False (!)  
d == np.nan         -> all False  
  
np.isnan(d)        -> [F F True F]
```

“Unknown” cannot equal “unknown” — you do not know whether the two missing values were the same. So `==` never finds a nan. You must use `np.isnan`.

! One nan poisons the whole calculation

```
d = np.array([1., 2., np.nan, 4.])  
d.sum()    -> nan  
d.mean()   -> nan  
d.max()    -> nan
```

Any arithmetic touching a nan produces a nan, so a single missing value turns an entire summary into nan. That is deliberate: NumPy refuses to quietly invent an answer. Your job is to decide what should happen instead.

Two ways to compute around a nan

1. The nan-aware functions

```
d = np.array([1., 2., np.nan, 4.])

np.nanmean(d)    -> 2.333
np.nanmedian(d)  -> 2.0
np.nansum(d)     -> 7.0
np.nanstd(d)     -> 1.247
np.nanmin  nanmax  nanpercentile
```

Each one skips the nans and works with what is left.

2. Filter them out yourself

```
clean = d[~np.isnan(d)]
clean      -> [1. 2. 4.]
clean.mean() -> 2.333

# ~ is 'not' — Day 12's boolean mask
# Same answer, and you can see
# exactly what was removed.
```

Prefer this when you want to report how many values you dropped.

Two edge cases that will surprise you

```
all_nan = np.array([np.nan, np.nan])
np.nansum(all_nan)  -> 0.0    (!)
np.nanmean(all_nan) -> nan + a warning
```

A sum of nothing is defined as 0, so nansum reports 0.0 for a column that is entirely missing — a real zero and a missing column look identical in your output. Always report the COUNT alongside any nan-aware summary.

Finding and counting what is missing

The three lines to run on any new dataset

```
np.isnan(d).sum()      how many missing
np.isnan(d).any()      any at all?
np.isnan(d).mean()     the PROPORTION

np.isnan(m).sum(axis=0) per column
```

isnan returns a boolean array, so every Day 12 and Day 13 trick applies: sum counts them, any asks the yes/no question, and mean gives the fraction because True counts as 1.

Then decide — and say so

- Drop the rows — fine if few, dangerous if many
- Fill with the mean or median — keeps the row, distorts the spread
- Leave them and use nan-aware functions
- Whichever you choose, record how many there were

Integer arrays cannot hold nan

```
np.array([1, 2, np.nan])      dtype float64
np.array([1, 2, np.nan], dtype=int)
-> ValueError
```

nan is a floating-point value — there is no integer bit pattern for “missing”. So the moment a column contains one nan, the whole column becomes float. If your integer IDs suddenly print as 1.0 and 2.0, a missing value is the reason.

4

Relationships

How two variables move together · 15 minutes

- `corrcoef`
- Reading `r`
- What it does not mean

Correlation — do two things move together?

```
hours = np.array([1., 2., 3., 4., 5.])
scores = np.array([52., 55., 61., 68., 74.])

np.corrcoef(hours, scores)
-> [[1.      0.9922]
     [0.9922 1.      ]]
```

corrcoef returns a MATRIX, not a number. The diagonal is always 1 — every variable correlates perfectly with itself. The answer you want is the off-diagonal element:

```
r = np.corrcoef(hours, scores)[0, 1]    -> 0.9922
```

Reading r

r	Means
+1	perfect: up together
+0.7	strong positive
0	no LINEAR relation
-0.7	strong negative
-1	perfect: one up, one down

⚠ Two things r does not tell you

It measures only STRAIGHT-LINE relationships. A perfect U-shaped curve can give r close to 0 — the relationship is real and strong, just not linear, so always plot before you trust a number.

And it says nothing about CAUSE. Ice cream sales correlate with drowning deaths; neither causes the other, because summer causes both. $r = 0.99$ is a reason to investigate, never a conclusion.

Debug corner — eight statistics bugs

You wrote	What happens	Fix
<code>d == np.nan</code>	all False — never matches	<code>np.isnan(d)</code>
<code>d.mean()</code> with a nan	returns nan	<code>np.nanmean(d)</code>
<code>np.nansum(all_nan_col)</code>	returns 0.0, looks like real data	report the count too
<code>np.mode(x)</code>	AttributeError — it does not exist	<code>np.unique(..., return_counts=True)</code>
<code>np.corrcoef(a, b)</code> as a number	it is a 2x2 MATRIX	<code>np.corrcoef(a, b)[0, 1]</code>
<code>arr.std()</code> as sample std	ddof=0, divides by n	<code>arr.std(ddof=1)</code>
mean on skewed data	one outlier drags it	report the median too
<code>np.array([1, 2, np.nan], dtype=int)</code>	ValueError	nan forces float

Rows 1 to 3 are all the same lesson: missing data does not announce itself. All eight are in today's practice notebook.

Putting it together — an honest summary report

```
import numpy as np

# marks for 6 students - two are missing
marks = np.array([88., 71., np.nan, 64., 95., np.nan])

def describe(a, name="data"):
    """Summarise an array honestly, missing values included."""
    missing = np.isnan(a)
    n_missing = missing.sum()
    clean = a[~missing]

    if clean.size == 0:
        # everything was missing
        print(f"{name}: no usable values")
        return

    q1, q3 = np.percentile(clean, [25, 75])
    print(f"{name}")
    print(f"  count    {clean.size} of {a.size}"
          f"    ({n_missing} missing)")
    print(f"  mean    {clean.mean():.2f}")
    print(f"  median  {np.median(clean):.2f}")
    print(f"  std     {clean.std(ddof=1):.2f}")
    print(f"  range   {clean.min():.0f} to {clean.max():.0f}")
    print(f"  IQR     {q3 - q1:.2f}")

describe(marks, "marks")
```



Count FIRST

how much data is real



Mask once

~missing reused for everything



Guard the empty case

all-missing does not crash



mean AND median

the gap reveals skew



ddof=1

sample std, stated explicitly



IQR too

spread an outlier cannot distort



Day 3's docstring

the habit never stopped mattering

Reporting the count is what makes this honest. A mean over 4 of 6 values is a different claim from a mean over all 6.

Practice session — 30 minutes



Predict the output

mean vs median, nan, percentiles, corrcoef

8 questions



Spot & fix the bug

== nan, poisoned mean, matrix r, ddof

4 questions



Write the code

Summarise, detect, fill, rank, correlate

5 tasks

Your notebook

[Day14_Practice_Questions.ipynb](#)

- Count the nans first
- Report mean AND median
- Raise a hand — I am walking around
- Solutions shared after class

Every summary you print today must say how many values it was computed from. A number without its count is not a finding.

TODAY IN TWELVE LINES

Recap

- 1 The mean uses every value; one outlier drags it a long way.
- 2 The median barely moves. Report both when they disagree.
- 3 `np.average` takes weights; `np.mean` does not.
- 4 There is no `np.mode` — build it from `np.unique`.
- 5 `std` and `var` default to `ddof=0`; `np.cov` defaults to `ddof=1`.
- 6 `np.percentile(a, 50)` is the median. $IQR = Q3 - Q1$.
- 7 `nan != nan`, so `== nan` never matches. Use `np.isnan`.
- 8 One `nan` makes any sum, mean or max return `nan`.
- 9 `np.nanmean` and friends skip them; so does `d[~np.isnan(d)]`.
- 10 `np.nansum` of an all-missing array returns `0.0`, not `nan`.
- 11 A `nan` forces the whole array to float — ints cannot hold it.
- 12 `corrcoef` returns a MATRIX; take `[0, 1]`. `r` is not cause.

Homework

Write `describe()` for a 2-D array, reporting per column · Find the outliers in a dataset using the 1.5 x IQR rule · Take a clean array, blank out three values, and compare every summary before and after

Next class — Topic 1.14

pandas: Series & DataFrames — the labelled table that everything from here runs on. NumPy underneath, names on top.